

Task 3 - Model Deployment Using FastAPI and Docker

1. Title

Model Deployment – Dogs vs Cats Image Classification API

2. Project Overview

This project focuses on deploying a trained Dogs vs Cats Convolutional Neural Network (CNN) model as a REST API using FastAPI.

The API accepts an image as input, preprocesses the image, sends it to the trained CNN model, and returns the predicted class along with its confidence score.

Docker is used to containerize the FastAPI application so that the model, application code, and required dependencies can run in a consistent environment.

3. Objective

The main objectives of this project are:

- Deploy a trained deep learning model as an API.
- Create an API endpoint for model inference.
- Accept an image file as input.
- Preprocess the image before prediction.
- Return the predicted class and confidence score.
- Containerize the API using Docker.
- Test the API using Swagger UI.

4. Technologies Used

- **Python** – Programming language
- **TensorFlow / Keras** – Used to load and run the trained CNN model

- **FastAPI** – Used to create the REST API
- **Uvicorn** – Used to run the FastAPI application
- **NumPy** – Used for image array processing
- **Pillow** – Used for image loading and preprocessing
- **Docker** – Used for containerization

5. Project Structure

task 3

```
|
|— app.py
|— dog_cat_model.keras
|— requirements.txt
|— Dockerfile
|— README.md
```

6. Trained Model

The project uses a previously trained Dogs vs Cats CNN model.

The model file is:

dog_cat_model.keras

The model accepts RGB images with the following input shape:

$128 \times 128 \times 3$

The trained model is loaded when the FastAPI application starts.

7. Image Preprocessing

Before sending an image to the model, the following preprocessing steps are performed:

1. The uploaded image is read.
2. The image is converted to RGB format.

3. The image is resized to 128×128 pixels.
4. The image is converted into a NumPy array.
5. Pixel values are converted to floating-point values.
6. Pixel values are normalized by dividing by 255.
7. A batch dimension is added.
8. The processed image is passed to the CNN model.

8. FastAPI Implementation

The API is implemented in:

app.py

FastAPI is used to create the REST API and handle image uploads.

The application loads the trained model:

dog_cat_model.keras

and provides three endpoints.

Home Endpoint

GET /

This endpoint confirms that the API is running.

Example response:

```
{  
  "message": "Dogs vs Cats Classification API is running"  
}
```

Health Check Endpoint

GET /health

This endpoint checks the health status of the API.

Example response:

```
{  
  "status": "healthy",  
  "model": "Dogs vs Cats CNN"  
}
```

Prediction Endpoint

POST /predict

This is the main model inference endpoint.

It accepts an image file, preprocesses it, sends it to the trained model, and returns the prediction.

Example response:

```
{  
  "prediction": "dog",  
  "confidence": 66.11  
}
```

9. Running the API Locally

The required Python libraries are installed using:

```
pip install -r requirements.txt
```

The FastAPI application is started using:

```
uvicorn app:app --reload
```

The application runs at:

```
http://127.0.0.1:8000
```

10. Swagger API Documentation

FastAPI automatically provides interactive API documentation using Swagger UI.

The documentation can be accessed at:

`http://127.0.0.1:8000/docs`

The Swagger interface displays the available API endpoints.

The POST `/predict` endpoint can be tested by selecting the endpoint, uploading an image, and executing the request.

Dogs vs Cats Image Classification API 1.0 OAS 3.1

[/openapi.json](#)

API for predicting whether an image is a dog or a cat.

default

GET	/	Home
GET	/health	Health
POST	/predict	Predict

11. Docker Containerization

Docker was used to package the FastAPI application, trained model, and required dependencies into a container.

The Docker configuration is provided in:

Dockerfile

The Dockerfile uses a Python base image and installs the required packages from requirements.txt.

The application and model are copied into the Docker image.

Port 8000 is exposed for the FastAPI application.

12. Building the Docker Image

The project directory was opened in Command Prompt:

```
cd "C:\study\InternShips\AI\task 3"
```

The Docker image was created using:

```
docker build -t dogs-cats-api .
```

The image was successfully built with the name:

```
dogs-cats-api:latest
```

Screenshot to add here:

Insert screenshot showing the successful Docker image build.

```
C:\study\InternShips\AI\task 3>docker build -t dogs-cats-api .
[+] Building 396.0s (12/12) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                  0.1s
=> => transferring dockerfile: 286B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.10-slim 6.2s
=> [auth] library/python:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                    0.1s
=> => transferring context: 2B                                       0.0s
=> [1/6] FROM docker.io/library/python:3.10-slim@sha256:31dd4d9529d02d7436659061cb7564cd4733fc90e5e152709a942d53382ec8d0 14.0s
=> => resolve docker.io/library/python:3.10-slim@sha256:31dd4d9529d02d7436659061cb7564cd4733fc90e5e152709a942d53382ec8d0 0.1s
=> => sha256:920e618c41a69e0f6d0174ef4e025bec30f31a33d2b5c0470169961424883b1e 249B / 249B 0.5s
=> => sha256:1f43bcf5ef119a40c99f7ce031eea771bd201bd859e4d0c4bb826f325b69b14e 13.91MB / 13.91MB 7.2s
=> => sha256:7b0c451348123348c3184b372ada72a6c9c55009bf718c05266ba088ba80461d 1.29MB / 1.29MB 2.1s
=> => sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774087db0d81151570244c8 29.83MB / 29.83MB 9.0s
=> => extracting sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774087db0d81151570244c8 2.4s
=> => extracting sha256:7b0c451348123348c3184b372ada72a6c9c55009bf718c05266ba088ba80461d 0.4s
=> => extracting sha256:1f43bcf5ef119a40c99f7ce031eea771bd201bd859e4d0c4bb826f325b69b14e 1.6s
=> => extracting sha256:920e618c41a69e0f6d0174ef4e025bec30f31a33d2b5c0470169961424883b1e 0.1s
=> [internal] load build context                                    4.9s
=> => transferring context: 39.72MB                                  4.8s
=> [2/6] WORKDIR /app                                             0.3s
=> [3/6] COPY requirements.txt .                                  0.1s
=> [4/6] RUN pip install --no-cache-dir -r requirements.txt      221.7s
=> [5/6] COPY app.py .                                           0.4s
=> [6/6] COPY dog_cat_model.keras .                              0.1s
=> exporting to image                                             152.5s
=> => exporting layers                                              114.4s
=> => exporting manifest sha256:994db1ea3658fee8601c03b1ca22616a291dd618e1479bdf13510666334e2113f 0.0s
=> => exporting config sha256:dab3f8d3e34ad947e1bf5bf93c8d9eb1a190fbd3840315c2ad2eeea2dfbee091 0.0s
=> => exporting attestation manifest sha256:b1e8b987399c3ba0ae88d7609ee4891a3a0eb4b1cf479565816762c0204ccd78 0.0s
=> => exporting manifest list sha256:ac1d7ed8c86153f28e1c2ad7b576e03160dc299cbf750896b085adc885192b78 0.0s
=> => naming to docker.io/library/dogs-cats-api:latest            0.0s
=> => unpacking to docker.io/library/dogs-cats-api:latest         37.8s
```

13. Running the Docker Container

The Docker container was started using:

```
docker run -d -p 8000:8000 --name dogs-cats-container dogs-cats-api
```

The running container was checked using:

```
docker ps
```

The container successfully ran with port mapping:

```
8000:8000
```

The API was then accessible at:

```
http://127.0.0.1:8000
```

Screenshot to add here:

Insert screenshot showing docker ps with dogs-cats-container running.

14. Example Request

The model inference endpoint is:

POST /predict

An image file was uploaded through Swagger UI.

Test image:

train_test.jpg

The request was processed by the Dockerized FastAPI application.

POST /predict Predict

Parameters

No parameters

Request body required

file * required
string

Choose File train_test.jpg

Execute

15. Example Response

The API returned:

```
{  
  "prediction": "dog",  
  "confidence": 66.11  
}
```

HTTP response:

200 OK

Therefore, the API successfully processed the input image and returned a prediction.

Screenshot to add here:

Insert screenshot of Swagger showing the uploaded image and the successful JSON response.

16. Result

The trained Dogs vs Cats CNN model was successfully deployed as a REST API using FastAPI.

The API successfully:

- Accepted an image file.
- Preprocessed the image.
- Loaded the trained CNN model.
- Performed image classification.
- Returned the predicted class.
- Returned the confidence score.

The application was also successfully containerized using Docker and tested inside a running Docker container.

For the test image `train_test.jpg`, the API returned:

Prediction : dog

Confidence : 66.11%

HTTP Status : 200 OK

17. Deliverables

The completed project contains:

app.py
dog_cat_model.keras
requirements.txt
Dockerfile
README.md

The project also includes the following demonstration:

- FastAPI Swagger UI
- Example image prediction
- Docker image
- Running Docker container
- Example API response

18. Conclusion

This project demonstrates how a trained deep learning model can be deployed as a REST API using FastAPI.

The API provides a simple model inference endpoint that accepts an image and returns the predicted class with its confidence score.

Docker was used to containerize the application, model, and dependencies, providing a consistent environment for running the deployed model.

The Dogs vs Cats model was successfully tested through the FastAPI Swagger interface and Docker container.